

SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY

COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Scenario-Based : 20%

QUESTIONS:

Total Marks: 50

Annexure A

Code of Conduct:

I **J. Sai Dheeraj [192311191]** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: J.Sai Dheeraj.

Annexure A

Scenario-based

[1] Debugging & Optimization of Symmetric Encryption (DES / 3DES Implementation)

A Python program implementing DES and 3DES encryption produces inconsistent ciphertext for identical inputs across multiple runs.

[A] Identify and fix logical and implementation errors in key scheduling and permutation steps.

[B] Optimize the code to reduce execution time for large input datasets (≥ 10 MB).

[C] Justify whether replacing DES/3DES with a modern cipher improves both security and performance.

ANS: Debugging & Optimization of Symmetric Encryption (DES / 3DES)

A. Debugging

Problems:

- Incorrect key scheduling (wrong key shifts or round keys).
- Errors in initial/final permutation tables.
- Incorrect S-box substitution.
- Wrong padding of plaintext.
- Inconsistent key or IV usage.

Fixes:

- Generate all 16 round keys correctly.
 - Verify IP and FP permutation tables.
 - Use standard DES S-boxes.
 - Apply PKCS#7 padding.
 - Use the same key (and IV for CBC mode) during encryption and decryption.
-

B. Optimization

- Use optimized cryptographic libraries (PyCryptodome/OpenSSL).
- Process data in chunks (e.g., 64 KB) instead of loading the entire file.
- Avoid repeated key generation.
- Use efficient bitwise operations.
- Parallelize independent block processing (ECB only).

Result: Faster execution and lower memory usage for files larger than 10 MB.

C. Replace DES/3DES?

Yes.

Algorithm	Security	Performance
DES	Very Low	Fast
3DES	Moderate	Slow
AES	Very High	Fast

Justification:

- DES (56-bit key) is insecure.
- 3DES is secure but slow because it encrypts three times.
- AES provides stronger security with significantly better performance.

[2] Performance Analysis of Block Cipher Modes of Operation

A Python script implements AES-like block cipher modes (ECB, CBC, CFB, OFB) but suffers from high latency and memory inefficiency.

[A] Debug incorrect padding and IV handling causing decryption failures.

[B] Refactor the code to minimize redundant computations and memory usage.

[C] Compare and optimize the implementation to determine the most efficient mode for real-time secure communication.

ANS: Performance Analysis of Block Cipher Modes

A. Debugging

Issues

- Incorrect padding removal.
- Random IV generated during decryption.
- IV not transmitted with ciphertext.
- Wrong block size handling.

Fixes

- Use PKCS#7 padding.
 - Store/send IV with ciphertext.
 - Use the same IV for decryption.
 - Validate padding before removal.
-

B. Optimization

- Reuse expanded AES keys.
 - Encrypt data block by block.
 - Minimize memory copying.
 - Stream data instead of loading the entire file.
-

C. Best Mode

Mode	Security	Speed	Recommendation
ECB	Poor	Fast	Not recommended
CBC	Good	Medium	Good for files
CFB	Good	Medium	Stream data
OFB	Good	Fast	Error-resistant
GCM	Excellent	Fast	Best(modern choice)

Recommendation: AES-GCM for real-time secure communication because it provides both encryption and authentication.

[3] Debugging RSA Cryptosystem Implementation

A Python-based RSA encryption system fails when encrypting large messages and occasionally produces incorrect plaintext after decryption.

[A] Identify issues in prime generation, key generation, and modular exponentiation.

[B] Optimize the algorithm using efficient techniques (e.g., fast exponentiation, Chinese Remainder Theorem).

[C] Evaluate the trade-offs between key size, security, and computational performance.

ANS: Debugging RSA Cryptosystem

A. Debugging

Problems

- Weak prime generation.
- Incorrect public/private key calculation.
- Wrong modular inverse.
- Integer overflow for large messages.

Fixes

- Use Miller-Rabin primality test.
- Generate large random primes.
- Use correct modular inverse.
- Split long messages into blocks before encryption.

B. Optimization

- Use Fast Modular Exponentiation (Square-and-Multiply).
- Use Chinese Remainder Theorem (CRT) for faster decryption.
- Cache repeated computations.

Result: Decryption becomes about 3–4× faster.

C. Trade-offs

Key Size	Security	Performance
----------	----------	-------------

1024-bit	Low	Fast
2048-bit	High	Good
3072-bit	Very High	Slower
4096-bit	Excellent	Slow

Recommendation: 2048-bit RSA offers a good balance of security and performance.

[4] Optimization and Security Analysis of Diffie-Hellman Key Exchange

A Python implementation of the Diffie-Hellman key exchange is vulnerable to performance bottlenecks and potential security flaws.

[A] Debug incorrect shared key generation due to improper modular arithmetic.

[B] Optimize exponentiation operations for large prime values.

[C] Extend the program to mitigate man-in-the-middle attacks and evaluate the performance impact of your enhancements.

ANS: Optimization and Security Analysis of Diffie-Hellman

A. Debugging

Problem

- Incorrect modular arithmetic.

Fix

Use

$$K = (g^b \bmod p)^a \bmod p$$

or

$$K = (g^a \bmod p)^b \bmod p$$

Both users must compute the same shared secret.

B. Optimization

- Use Fast Modular Exponentiation.
- Use Python's built-in:

`pow(base, exponent, modulus)`

This is much faster than `(base**exponent) % modulus`.

C. Prevent Man-in-the-Middle Attack

Threat

- An attacker replaces exchanged public keys.

Solution

- Digital signatures
- Certificates
- Authenticated Diffie-Hellman
- TLS

Performance Impact

- Slight increase in computation.
- Significant improvement in security.

[5] Debugging and Enhancing Elliptic Curve Cryptography (ECC) Implementation

A Python script implementing Elliptic Curve Cryptography for key exchange produces incorrect results for certain curve parameters.

[A] Identify and fix errors in point addition and scalar multiplication logic.

[B] Optimize the implementation using efficient algorithms (e.g., double-and-add).

[C] Compare the optimized ECC implementation with RSA in terms of computational efficiency and security strength, and justify suitability for resource-constrained environments.

ANS: Debugging and Enhancing ECC

A. Debugging

Problems

- Incorrect point addition.
- Wrong point doubling.
- Improper handling of point at infinity.
- Incorrect modular inverse.

Fixes

- Verify elliptic curve equations.
 - Handle special cases correctly.
 - Perform all calculations modulo p .
-

B. Optimization

Use the **Double-and-Add Algorithm**.

Advantages:

- Reduces scalar multiplication complexity.
 - Much faster than repeated addition.
 - Standard method used in ECC implementations.
-

C. ECC vs RSA

Feature	ECC	RSA
Key Size	256-bit	3072-bit
Security	Very High	Very High
Speed	Faster	Slower
Memory	Low	High
Power Consumption	Low	High
IoT Suitability	Excellent	Poor

Recommendation: ECC is the preferred choice for resource-constrained devices such as IoT, smart cards, and mobile devices because it provides equivalent security with much smaller keys, lower computation, reduced memory usage, and lower power consumption than RSA.